

Vite & Gourmand — Site Traiteur Bordeaux

Projet ECF — TP Développeur Web et Web Mobile (Studi)

Antoine Banzet · 2026

Présentation

Site vitrine et de commande pour **Vite & Gourmand**, traiteur à Bordeaux depuis 1999. Permet aux visiteurs de consulter les menus, aux clients de passer commande et de déposer un avis. Inclut un espace employé (gestion des menus / plats / horaires / commandes / avis) et un espace administration (gestion des comptes employés, statistiques).

 Choix techniques et justifications détaillées : voir le **dossier technique** (document séparé).

Stack technique

Couche	Technologie
Frontend	HTML5 · CSS3 (custom properties) · JavaScript vanilla (aucun script inline, CSP stricte)
Backend	PHP 8.3 · POO / MVC · PDO · API REST JSON (sans framework, autoload PSR-4 Composer)
Base SQL	MySQL 8+ · InnoDB · utf8mb4
Base NoSQL	MongoDB (statistiques admin uniquement)
Authentification	Sessions PHP (cookie <code>HttpOnly</code>) · bcrypt cost 12
Emails	<code>mail()</code> natif en local · API Brevo en production

Structure du projet

```
projet/
├─ Dockerfile · docker-entrpoint.sh    Image de déploiement (php:8.3-apache)
├─ .htaccess                          Routage /api/* + en-tête Content-Security-Policy
├─
├─ frontend/                          Vue : pages HTML + CSS + JS
│   ├── index.html · menu.html · commande.html · panier.html · contact.html
│   ├── MonCompte.html · SeConnecter.html · SInscrire.html · reset-password.html
│   ├── employe.html · employe-dashboard.html · admin.html · admin-dashboard.html
│   ├── cgv.html · rgpd.html · legal.html · accessibilite.html    Pages légales
│   ├── style.css                      Feuille de styles unique
│   ├── js/
│   │   ├── navbar-inject.js          Génère et injecte le HTML de la navbar
│   │   ├── navbar.js                 Burger, état connecté, badge panier
│   │   ├── panier.js                 Utilitaires panier partagés (localStorage)
│   │   ├── actions.js                Liaison data-action → fonctions (remplace les onclick)
│   │   └─ pages/<page>.js            Script propre à chaque page (aucun JS dans le HTML)
│   ├── images/                       Assets visuels
│   └─ vite_et_gourmand.sql           Schéma SQL + données de test
├─ api/                               Backend PHP orienté objet
│   ├── index.php                     Front controller : autoload + config + routes → Kernel
│   ├── routes.php                    Table des routes (MÉTHODE chemin → Contrôleur::méthode)
│   ├── composer.json                 Autoload PSR-4 (namespace App\ → src/)
│   ├── .env.example                  Variables d'environnement (modèle)
│   ├── aiven-ca.pem                  Certificat SSL MySQL (production Aiven)
│   └─ src/
│       ├── Http/                     Kernel, Router, Request, JsonResponse, HttpException
│       ├── Controllers/               Controller (abstraite) + 11 contrôleurs
│       ├── Models/                    Commande, Utilisateur, Avis (règles métier)
│       ├── Enum/                      StatutCommande, StatutAvis, Role
│       ├── Repositories/              Repository (abstraite) + 7 repositories (tout le SQL)
│       ├── Auth/ · Security/ · Support/ · Mail/ · Mongo/           Services
│       ├── Database.php                Connexion PDO unique
│       └─ Config.php                  Chargement .env / variables d'environnement
├─ docs/
│   ├── *.pdf                         Dossier technique, manuel, charte, gestion de projet
│   ├── diagrammes/
│   │   └─ sources/                   Sources Mermaid + générateur des diagrammes de classes
│   └─ maquettes/                     Wireframes et mockups
```

Installation locale

Prérequis

- PHP 8.1+ (8.3 en production) et Composer

- MySQL 8+
- Serveur web avec `mod_rewrite` activé (Apache/Nginx), ou XAMPP / WAMP / Laragon
- Extension PHP `mongodb` (optionnelle — sans elle, seul le graphique admin est désactivé)

1. Base de données

Le fichier `vite_et_gourmand.sql` crée lui-même la base (`DROP` puis `CREATE DATABASE`) : il faut donc l'importer **sans** préciser de base de données.

```
mysql --default-character-set=utf8mb4 -u root -p < projet/frontend/vite_et_gourmand.sql
```

Le paramètre `--default-character-set=utf8mb4` est important : sans lui, l'import peut corrompre les caractères accentués (ex : « Entrée » devient « EntrÃ©e »).

2. Configuration backend

```
cp projet/api/.env.example projet/api/.env
cd projet/api && composer install      # génère l'autoload PSR-4 (aucune dépendance tierce)
```

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=vite_et_gourmand
DB_USER=root
DB_PASS=votre_mot_de_passe

MAIL_FROM=antoinebaban@gmail.com
MAIL_NAME=Vite & Gourmand
APP_URL=http://localhost/ViteEtGourmand/projet/frontend

# MongoDB – laisser vide pour désactiver le graphique admin (l'app fonctionne sans)
MONGO_URI=
MONGO_DB=vite_et_gourmand_logs

# Brevo – laisser vide en local : le code utilise mail() nativement
BREVO_API_KEY=
```

En production, `MAIL_FROM` doit être une adresse vérifiée comme expéditeur dans le compte Brevo (Expéditeurs, domaines et IP dédiée). Le domaine `viteetgourmand.fr` étant fictif, une adresse personnelle vérifiée est utilisée à sa place — le nom affiché reste « Vite & Gourmand » via `MAIL_NAME` . Sans expéditeur vérifié, Brevo rejette l'envoi silencieusement (aucune erreur remontée côté application).

3. Initialisation des mots de passe

Les mots de passe importés par `vite_et_gourmand.sql` sont des marqueurs (`$2y$12$PLACEHOLDER_RUN_SETUP`), pas de vrais hachages — il faut les générer une fois localement.

Pour chaque mot de passe de test, générer son hachage bcrypt :

```
php -r "echo password_hash('Admin2026!', PASSWORD_BCRYPT, ['cost'=>12]);"
```

Puis, dans phpMyAdmin (ou `mysql`), coller le hachage obtenu :

```
UPDATE utilisateur SET password = '<hash copié>' WHERE email = 'admin@viteetgourmand.fr';  
UPDATE utilisateur SET password = '<hash copié>' WHERE email = 'employe@viteetgourmand.fr';  
UPDATE utilisateur SET password = '<hash copié>' WHERE email IN  
('jean.martin@email.fr', 'sophie.b@email.fr', 'pierre.d@email.fr');
```

(Les trois comptes clients partagent le même mot de passe `Client2026!`, donc le même hachage.)

4. Lancer le projet

Ouvrir `http://localhost/ViteEtGourmand/projet/frontend/index.html`

Comptes de test

Rôle	Email	Mot de passe
Administrateur	admin@viteetgourmand.fr	Admin2026!
Employé	employe@viteetgourmand.fr	Employe2026!
Client 1	jean.martin@email.fr	Client2026!
Client 2	sophie.b@email.fr	Client2026!
Client 3	pierre.d@email.fr	Client2026!

Endpoints API principaux

POST	/api/auth/login	
POST	/api/auth/register	
POST	/api/auth/logout	
GET	/api/auth/me	
POST	/api/auth/forgot-password	
POST	/api/auth/reset-password	
GET	/api/menus	filtres : theme_id, regime_id, prix_min, prix_max, personnes, q
GET	/api/menus/populaires	
GET	/api/menus/{id}	
POST	/api/commandes	
GET	/api/commandes/mes-commandes	
GET	/api/commandes	employé / admin
PUT	/api/commandes/{id}	
PUT	/api/commandes/{id}/statut	employé / admin
PUT	/api/commandes/{id}/annuler	
GET	/api/commandes/{id}/historique	
GET	/api/avis	
POST	/api/avis	
PUT	/api/avis/{id}/validation	employé / admin
GET	/api/admin/stats	
GET	/api/admin/utilisateurs	
POST	/api/admin/employes	
PUT	/api/admin/employes/{id}/desactiver	admin
PUT	/api/admin/employes/{id}/activer	admin
POST	/api/contact	
POST	/api/devis	
POST	/api/factures	demande de facture (client)

Déploiement en production

Architecture : **Render** (PHP via Docker) + **Aiven** (MySQL) + **MongoDB Atlas** + **Brevo** (emails). Démarche complète et justifications : voir le dossier technique, section Déploiement.

- Application en ligne : <https://ecf-dwwwm.onrender.com>
- Dépôt GitHub : <https://github.com/antoinebaban-sketch/ECF-DWWW>

Organisation Git

Branche	Usage
main	Code stable / production
develop	Intégration des fonctionnalités
feature/*	Une branche par fonctionnalité, fusionnée dans develop (--no-ff) après test

Chaque fonctionnalité part de `develop` sur une branche dédiée ; `develop` est fusionnée dans `main` pour les versions stables. Messages de commit atomiques et préfixés (`feat` , `fix` , `refactor` , `docs`).

Projet réalisé dans le cadre du TP Développeur Web et Web Mobile — Studi 2026.